

Turbin3 Capstone, Deliverable 1: Heirloom

Project: Heirloom, non-custodial digital succession for Solana wallets.

Part 1: Final Project Proposal & LOI

Project Overview

Heirloom is a non-custodial inheritance protocol for Solana wallets. An owner names one or more beneficiaries and sets a check-in interval that proves they're still active. A missed check-in doesn't trigger anything immediately, the plan moves through a staged series of warnings first, giving the owner multiple chances to check in before any funds move. Only after the final deadline passes does a transaction sweep the owner's balance to the named beneficiary or beneficiaries, and at no point does the owner deposit assets into an escrow, a vault, or a shared keypair to make that possible.

Value Proposition

An owner's SOL and tokens stay in their own wallet and accounts for as long as they're alive and checking in, no deposit step, no shared keypair, no separate app-controlled account holding their funds. If a check-in is missed, the plan escalates through warning stages rather than firing on a single missed deadline, so ordinary things like travel, a lost phone, or a hospital stay don't turn into a premature transfer. Only once the final deadline genuinely passes does the sweep execute, and it moves whatever the owner's live balance is at that moment, not an amount frozen back when the plan was set up, which means the owner keeps completely normal use of their funds right up until the trigger actually fires. Inheritance should be this simple: assets move from the owner to the beneficiary when the condition is met, with no added custody layer in between.

Product-Market Fit

No product on Solana lets an owner do this without giving something up first. Eternal Key and Dead Man's Vault both require depositing assets into a program-owned vault ahead of time, meaning the owner loses normal use of those specific assets the whole time the switch is armed. Deadhand Protocol avoids a vault, but only by holding a shard of the owner's key on a server, a different kind of third-party trust. Heirloom's fit is narrower and more defensible than "no inheritance product

exists": it's the only approach found across a live competitor search that keeps the owner's assets in their own wallet, under their own key, with no third party holding so much as a fragment of it, while still guaranteeing an automatic transfer on death or incapacitation. That specific gap, not inheritance in general, is the product.

Target Markets & User Profiles

Market segments, in priority order:

1. **Beachhead: long-term self-custody Solana holders** already active in existing crypto communities. Most likely to already feel this pain and reachable without needing a distribution partner first.
2. **Crypto-native families**, where one person manages the wallet but a spouse, kids, or parents would need the assets someday.
3. **Aging or health-risk holders** who want continuity without setting up a formal will or trust for digital assets.
4. **Retail solo holders with no existing legal will** covering crypto specifically.

DAOs and multisig treasury succession were considered as a segment and deprioritized, the operational and governance requirements are different enough from a personal inheritance plan that they'd dilute a first version rather than strengthen it.

User profiles:

1. **The Owner.** A self-custody Solana user, technical enough to manage their own wallet, wants continuity for their family without giving up day-to-day control.
2. **The Beneficiary.** Usually a family member, not necessarily crypto literate. Receives assets automatically, without needing to sign anything for SOL, and only needs an existing associated token account for SPL assets.
3. **The Relay/Keeper.** A permissionless operational actor, not a customer, that holds the owner's presigned SOL sweep and submits it, along with the token distribution, once the deadline has genuinely passed.

Competitor Landscape

Competitor	Chain	Mechanism	How Heirloom differs
Eternal Key	Solana	Owner deposits assets into an escrow account ahead of time.	No deposit, assets stay in the owner's own wallet.

Competitor	Chain	Mechanism	How Heirloom differs
Dead Man's Vault (DMV)	Solana (Seeker)	Heartbeat check-in with a 4-stage escalation, owner deposits SOL/SPL/NFTs into a vault PDA up front.	Same escalation idea, but no vault deposit, uses a presigned transaction and a token delegate instead of custody.
Deadhand Protocol	Multi-chain (incl. Solana)	Splits the owner's seed phrase into shards (Shamir's Secret Sharing), a server holds one shard.	No server ever holds any piece of the owner's key.
Sarcophagus	Ethereum + Arweave	Dead man's switch for encrypted data/document release, DAO-run, VC-funded.	Different chain and use case (data release vs. live wallet-balance transfer), tracked as an adjacent reference point rather than a direct competitor.

Founder-Market Fit

I'm a generalist engineer with 4 years of professional experience, most of it in SRE, with solid experience across mobile app development, full-stack development, and DevOps, including freelance work building Solana Mobile demo applications. I got into crypto in April 2025. What I bring to this specific project is an infrastructure and reliability mindset: while building the proof of concept for Heirloom's core mechanism, I found a real, non-obvious fragility (a presigned transaction becomes a single-use bearer instrument under Solana's durable-nonce rules) before it ever reached production, exactly the kind of adversarial-conditions thinking this protocol needs, given it will hold or move other people's inheritance on a single missed check-in. I don't have deep pre-existing Solana or Anchor security expertise, and I'm building that directly through this project rather than claiming it up front. My current biggest gap is distribution and network: no existing relationship yet with wallet teams (Phantom, Solflare, Backpack) or Solana community channels that this product will eventually need to reach the families it's meant for. Closing that is an explicit next step, not something being ignored.

Actors

Direct Actors (signers who invoke instructions):

- **Owner.** Creates the plan, sets up the durable nonce account for SOL, approves the program as a token delegate for SPL assets, checks in and re-signs a fresh presigned SOL sweep each cycle, updates beneficiaries, and can revoke or close the plan while it's still active. Never deposits assets anywhere.

- **Relay/Keeper.** A permissionless caller that holds the owner's presigned SOL sweep, advances warning stages, and submits the distribution once the deadline has genuinely passed, checking the plan's on-chain state itself before ever broadcasting. Signs transactions but never gains value from them.
- **Protocol Admin.** A small multisig or governance PDA that owns global protocol configuration. Signs only for protocol-level actions like pausing new plan creation in an emergency.

Beneficiaries (gain value without directly signing):

- **Beneficiary.** Named by the owner, receives SOL or tokens once the plan's deadline passes and someone else submits the distribution. No signature needed to receive SOL, and only an existing associated token account is needed for tokens.

Administrators:

- **Protocol Config PDA.** Holds global parameters (minimum and maximum check-in interval, pause flag).

Stakeholders (external parties with interest, not on-chain signers):

- **Wallet providers** (Phantom, Solflare, Backpack). Adoption depends on these surfacing check-in reminders and plan status to users.
- **Estate planning and legal advisors.** Interested in whether an on-chain transfer holds up against real-world probate law.
- **Other legal heirs.** Family members not named on-chain who may have a real-world legal claim conflicting with an automatic on-chain transfer.
- **Security researchers and auditors.** Have a direct interest given the project's own POC already found one non-obvious fragility before mainnet.

Use Cases

Each use case is one atomic state transition mapped to one Anchor instruction handler.

1. Create Inheritance Plan - Handler: `initialize_plan` - Actor: Owner (signer) -

Preconditions: no existing plan PDA for this owner. - State transition: no plan account exists, to a new Plan PDA created holding the owner's pubkey, the check-in interval, the beneficiary list with share splits, `last_checkin` set to the current Clock timestamp, and status set to Active.

2. Register Nonce Account - Handler: `register_nonce` - Actor: Owner (signer) - Preconditions: plan exists and is Active, signer matches `plan.owner`, the nonce account already exists (created via

the standard System Program, nonce authority held by the owner) and holds only its own rent-exempt balance, no inheritance assets. - State transition: the nonce account's pubkey is recorded on the Plan PDA, so `heir_sweep` can later verify it's being submitted against the expected nonce account. No SOL belonging to the owner's estate moves anywhere.

3. Approve Token Delegate - Handler: `approve_token_delegate` - Actor: Owner (signer) - Preconditions: plan exists and is Active. - State transition: the delegate field on the owner's own SPL token account is set to the Plan PDA, with a delegated amount equal to the balance enrolled. The tokens never leave the owner's own account.

4. Check In - Handler: `check_in` - Actor: Owner (signer) - Preconditions: plan exists, signer matches `plan.owner`. - State transition: `plan.last_checkin` updates to the current Clock timestamp, and status resets to Active if it had moved into a warning stage. Alongside this call, the owner advances their own nonce account and re-signs a fresh presigned `heir_sweep` transaction for the relay. A stale copy's embedded nonce no longer matches, so submitting it later fails inside `AdvanceNonceAccount` itself, the one failure mode that doesn't burn anything, so old copies simply stop working.

5. Update Beneficiaries - Handler: `update_beneficiaries` - Actor: Owner (signer) - Preconditions: plan is Active, deadline has not yet passed. - State transition: the beneficiary list and share splits stored on the plan are replaced with the new set.

6. Revoke and Close Plan - Handler: `close_plan` - Actor: Owner (signer) - Preconditions: plan is Active, deadline has not yet passed. - State transition: any delegated token approval is reset to zero, the nonce account is closed and its rent reclaimed by the owner, the Plan PDA is closed and its rent reclaimed. There's no vault to unwind, since the owner's assets never left their own accounts.

7. Advance Warning Stage - Handler: `advance_warning_stage` - Actor: Relay/Keeper (signer, permissionless) - Preconditions: `Clock >= plan.last_checkin + plan.checkin_interval + stage_offset`, and the plan's current stage is earlier than the stage being advanced to. - State transition: `plan.status` moves from Active to Warning Stage N, or from one warning stage to the next.

8. Distribute SOL - Handler: `heir_sweep` - Actor: Relay/Keeper (signer, submits the owner's presigned durable-nonce transaction); Beneficiary receives passively. - Preconditions: the transaction is the owner's presigned `[AdvanceNonceAccount, heir_sweep(beneficiary)]`, the embedded nonce matches the registered nonce account's current value, `Clock` is past the final deadline, plan has not already been distributed. - State transition: `AdvanceNonceAccount` executes, then `heir_sweep` reads the owner's live lamport balance and CPIs `system_program::transfer(owner, beneficiary, amount)` for that live amount. `plan.status` becomes Distributed. A premature submission fails this whole transaction and still burns the nonce, which is why the relay is expected to verify the deadline itself before broadcasting.

9. Distribute Tokens - Handler: `distribute_tokens` - Actor: Relay/Keeper (signer, permissionless); Beneficiary receives passively into an existing associated token account. - Preconditions: same deadline condition as use case 8, delegate approval on the owner's token account is still valid. - State transition: tokens move from the owner's own token account to each beneficiary's token account via `transfer_checked`, using the Plan PDA's delegate authority, computed from the owner's live token balance at execution time.

10. Set Protocol Paused - Handler: `set_protocol_paused` - Actor: Protocol Admin (signer) - Preconditions: signer matches `protocol_config.admin`. - State transition: `protocol_config.paused` flips true or false. While paused, `initialize_plan` and `register_nonce` are blocked for new plans, but check-ins, warning-stage advances, and distributions for plans already in flight are unaffected.

Part 2: Process Appendix (Red Team Log)

This section shows the work behind Part 1: the original drafts, the AI critique run against them, and what was accepted, rejected, or left open, with the reasoning for each.

Original Drafts (before AI critique)

Value Proposition & PMF, as first written:

Idea is inheritance without giving up access to assets. Assets inside a wallet need to be passed on to a dedicated account/multiple accounts declared by the user, user decides how often a checkin is performed to verify user still lives, if user breaches that amount, a tx takes place which moves the assets from that account to the beneficiary/ies. Right now no product exists on Solana which allows transfer of assets without taking those assets into an escrow, or having a shared key-pair. Inheritance should be simple, A goes to B. Thats the solution I am aiming for. The product is fit for everyone who wishes to pass on their crypto assets.

Target Market, as first written:

Its meant to target everyday users, and their families who they can give a separate wallet key to, same public key can be added as beneficiary, the family doesnt need to be tech literate to

be able to claim it as that would happen behind the scenes, all they need is access to the beneficiary wallet.

Competitor Landscape, as first written:

Competitors like eternal-key depend on assets being locked in escrows. Making it unusable for the person who intends to hand down the assets. That alone is a massive issue for anyone who wants to be able to enjoy their own assets as well.

Only one competitor had been identified at this stage, from a single GitHub link, with no check against a raises tracker, a hackathon project explorer, or social chatter.

Founder-Market Fit, as first written:

I am a generalist engineer, with 4 YoE as a professional. Most experienced in SRE, but have ample experience in Mobile App Development, Full stack development and Devops, most notably working as a freelancer for Solana Mobile Demo applications. Jumped into crypto April of 2025. My motivation to build this is, that common sense ideas and solutions should exist already without complications, if they dont, thats an issue and makes mass adoption unlikely.

Round 1: AI Red Team on Value Prop, PMF, Market Segments, and Competitors

#	Attack	Accepted or rejected	Why, and what changed
1	The check-in mechanism is a liveness oracle problem: a missed check-in from travel, a lost phone, or a hospital stay isn't death, and sweeping a live wallet on a single missed deadline could be worse than the problem being solved.	Accepted	Added a staged warning escalation before any sweep executes, borrowed from Dead Man's Vault's pattern, instead of one hard deadline. Now use case 7 in Part 1.
2	Sweeping the <i>live</i> balance at execution time, not a fixed amount, makes a beneficiary's actual inheritance unpredictable.	Accepted, as a disclosure, not a redesign	This is the direct consequence of the design choice that lets the owner keep normal control, so it was kept and stated explicitly in the Part 1 value proposition rather than left implicit.

#	Attack	Accepted or rejected	Why, and what changed
3	"No product exists on Solana which allows transfer without escrow or shared key" was proven false by a search that found Dead Man's Vault.	Accepted	PMF narrowed in Part 1 from "no product exists" to "no product avoids a vault deposit while keeping non-custodial guarantees."
4	"Inheritance should be simple, A to B" is a slogan, not a validated need. No one's been asked whether check-in friction or beneficiary UX is the real adoption blocker.	Accepted, left open	No claim was changed to paper over this. Real user outreach in Solana communities is still a pending action item, not yet done.
5	"Fit for everyone who wishes to pass on their crypto assets" is too broad to function as a market segment.	Accepted	Replaced with the prioritized segment list and named beachhead in Part 1.
6	"The family doesn't need to be tech literate" glosses over the fact that someone still has to get the beneficiary a wallet and explain what's happening.	Accepted, as a scoped product gap	Kept as a known onboarding and support burden. Not solved yet, not hidden either.
7	Only one competitor had been named, from a single GitHub link, with no real search done.	Accepted	Ran live research and found two more direct Solana competitors (Dead Man's Vault) and one cross-chain adjacent one (Sarcophagus), plus Deadhand Protocol. Full table now in Part 1.
8	Deadhand's server-assisted key-sharding approach solves an adjacent problem without any smart contract at all, a legitimate substitute threat, not a "different approach, doesn't count" dismissal.	Accepted, tracked	Kept in the final competitor table as a distinct threat with a different trust model (a server holding a key shard), rather than dismissed.

Round 2: AI Red Team on Founder-Market Fit

Attack	Accepted or rejected	Why, and what changed
Four years of generalist engineering experience with an SRE focus is not specialized Solana or Anchor security experience, and under 18 months of crypto exposure is thin for a protocol moving other people's savings on a single missed check-in.	Accepted	The final FMF in Part 1 no longer implies pre-existing Solana expertise. It states plainly that Solana- and Anchor-specific expertise is being built through this exact project.

Attack	Accepted or rejected	Why, and what changed
No stated relationship with wallet teams, the Solana Foundation, or legal/estate-planning partners, all of whom are needed to reach the "everyday families" segment.	Accepted	Named directly in Part 1's FMF as the current biggest gap and an explicit next step, rather than omitted.
The founder's own POC surfaced a real security flaw that wasn't anticipated going in.	Accepted, reframed	Instead of treating this as a liability to downplay, Part 1's FMF leads with it as evidence of the infrastructure/reliability rigor the project needs.

Round 3: A Design Reversal Caught Mid-Process (Vault Deposit)

This is the clearest piece of proof-of-work in this log, an actual course correction, not just a critique that was agreed with in the abstract.

What happened: an early draft of the Phase 2 use cases, written to route around the fragility found in the POC (below), defaulted to a native-SOL vault deposit as "the pragmatic default", the owner would deposit SOL into a program-owned vault PDA, exactly the same pattern used by Eternal Key and Dead Man's Vault.

The catch: flagged directly during review: "we are avoiding this in our build." A vault deposit isn't a neutral technical fallback here, it directly contradicts the PMF in Part 1, which exists specifically because Eternal Key and Dead Man's Vault require that exact deposit step.

The reversal: the vault-based use case (`deposit_sol`) was removed outright and replaced with `register_nonce` , no funds move. The SOL side of the design was rebuilt around the presigned durable-nonce mechanism from the POC instead, and its known fragility (below) is now managed operationally rather than by giving up the no-deposit property: a single trusted relay holds the presigned bytes instead of publishing them or handing them to the beneficiary, the relay checks the plan's on-chain state before ever broadcasting, and every check-in rotates the nonce and has the owner re-sign a fresh copy, so stale copies fail harmlessly instead of sitting around as a live risk.

Before:

```
Handler: deposit_sol
State transition: lamports move from the owner's wallet into the plan's vault PDA.
```

After:

Handler: register_nonce

State transition: the nonce account's pubkey is recorded on the Plan PDA. No SOL below to the owner's estate moves anywhere.

POC Technical Validation Log

Hypothesis: a presigned durable-nonce transaction with no amount parameter in its instruction data could let a beneficiary later sweep whatever the owner's balance happens to be, without the owner needing to be present, and without ever freezing an amount at signing time. This ran against the initial intuition that presigning a variable amount isn't possible on Solana, since a signature commits to fixed instruction data.

Test: built a working proof of concept against a real `solana-test-validator` run (`poc-heir-sweep/`, full writeup in `poc-heir-sweep/README.md`).

Confirmed: the mechanism works exactly as designed. A durable-nonce transaction signed once, held untouched, and submitted for the first time after a timeout sweeps the owner's live balance, not the balance that existed at signing time, even when that balance changed in between. This corrected an earlier assumption in this project's own drafts that presigning couldn't work at all.

Found, empirically: a single premature submission attempt, by anyone holding the transaction bytes, not just the beneficiary, and even though it fails, permanently destroys that transaction. Solana's durable nonce accounts advance on any failed attempt, unless the failure occurs inside the `AdvanceNonceAccount` instruction itself, specifically to prevent free replay. That makes the one valid copy of the owner's authorization a single-use bearer instrument.

Consequence for the design: this finding is what forced the Round 3 decision above. Rather than accept it as a reason to fall back to a vault, the accepted mitigation was built specifically around the one safe failure mode this test found, a mismatched nonce fails inside `AdvanceNonceAccount` without burning anything, which is why check-in now rotates the nonce on every cycle instead of leaving one presigned copy valid indefinitely.

Sources

- [Eternal Key](#) / [GitHub](#)
- [Dead Man's Vault \(DMV\)](#), [GitHub](#)
- [Deadhand Protocol](#)

- [Sarcophagus overview](#), [Decrypt](#)
- [poc-heir-sweep/](#) — POC code and validator run